Workshop on Computational Fluid Dynamics (CFD) with OpenFOAM IMP-PAN Gdańsk

Tutorial #2: Water-air Ejector

Multiphase Transient Flow

Robert Castilla
CATMech - Universitat Politècnica de Catalunya
Department of Fluid Mechanics

2025-26
Version 1.0

Contents

Learning Objectives

By the end of this tutorial the student will be able to

1. **Understand** basic differences between Volume Of Fluid and Euler-Euler approaches
2. **Run** a simulation of a water ejector with VOF and EE approaches
3. **Compute and analyze** entrainment ratio M
4. **Visualize** basic results of multiphase ejector with `paraview`

1 Introduction and Theoretical Background

In this section a water-air ejector will be simulated. A water-air ejector is a simple device that uses high speed water to provoke pressure depression and entrain air from another inlet. Both water and air are mixed in the mixing chamber and expelled through a diffuser to recover the pressure lost.

Two approaches will be explored to make this simulation.

The first approach is the **Volume Of Fluid (VOF)** method. The equations are basically the same as in the previous session, and the interface is tracked with the phase fraction $\alpha \in [0, 1]$,

$$\frac{\partial(\rho\mathbf{U})}{\partial t} + \nabla\cdot(\rho\mathbf{U}\mathbf{U}) = -\nabla p + \nabla\cdot\left[\mu\left(\nabla\mathbf{U} + \nabla\mathbf{U}^T\right)\right] + \rho\mathbf{g} + \mathbf{F}_\sigma \tag{1}$$

with the equation of the phase fraction transport

$$\frac{\partial\alpha}{\partial t} + \nabla\cdot(\alpha\mathbf{U}) = 0$$

and the mixture properties in each cell

$$\rho = \alpha\rho_w + (1-\alpha)\rho_a, \qquad \mu = \alpha\mu_w + (1-\alpha)\mu_a$$

Surface tension force is computed with the Continuum Surface Force (CSF) model,

$$\mathbf{F}_\sigma = \sigma\kappa\nabla\alpha, \qquad \kappa = -\nabla\cdot\left(\frac{\nabla\alpha}{|\nabla\alpha|}\right)$$

Alternatively, the second approach considered here is the **Euler-Euler** method, where each phase is treated as interpenetrating continua, with interphase terms for momentum, heat and mass transfer. In this case, several phases can be included, with phase fractions α_k,

$$\sum_{k=1}^{N} \alpha_k = 1$$

For each phase, continuity has to be fulfilled,

$$\frac{\partial(\alpha_k \rho_k)}{\partial t} + \nabla \cdot (\alpha_k \rho_k \mathbf{U}_k) = 0 \tag{2}$$

as well as the momentum equation,

$$\frac{\partial(\alpha_k \rho_k \mathbf{U}_k)}{\partial t} + \nabla \cdot (\alpha_k \rho_k \mathbf{U}_k \mathbf{U}_k) = -\alpha_k \nabla p + \nabla \cdot \left[\alpha_k \mu_{k,\text{eff}} \left(\nabla \mathbf{U}_k + \nabla \mathbf{U}_k^T\right)\right] + \alpha_k \rho_k \mathbf{g} + \mathbf{M}_k \tag{3}$$

where $\mathbf{M}_k$ is the interphase momentum transfer, that includes terms for drag, lift, virtual mass and turbulent dispersion.

Table 1: Comparison of VOF and Euler-Euler approaches

Feature	**VOF**	**Euler-Euler**
Interface representation	Sharp (captured)	Diffuse (interpenetrating)
Momentum equations	One (mixture)	One per phase
Interphase forces	Surface tension only	Drag, lift, virtual mass, turbulent dispersion
Turbulence	Single model (mixture properties)	Per-phase models
Best for	Stratified, free-surface flows	Dispersed, bubbly, particle-laden flows
Computational cost	Lower	Higher

1.1 Case Overview

2 Case Setup

2.1 Python virtual environment

To facilitate the generation of the mesh with `blockMesh` we are going to use a python code. It is convenient to create first a user virtual environment, so that packages can be locally installed. Here follows the procedure

```
# Create the virtual environment. Here the name is 'my_env' but it can be any
    name
python -m venv my_env
# Activate the virtual environment
source ./my_env/bin/activate
# From now on a '(my_env)' will be shown with prompt
# To deactivate the virtual environment, just type
deactivate
```

The module ofblockmeshdicthelper, by A. Takaaki will be used. It has to be installed in the environment with

```
pip install 'ofblockmeshdicthelper @git+https://github.com/takaakiaoki/
    ofblockmeshdicthelper.git'
```

If some other module is needed, just install it with **`pip install <module name>`**.

3 Part A: Volume of Fluid Approach

The clean case can be downloaded from the GitHub repository,

3.1 Running the Simulation

The simulation process is split into two automated phases: mesh generation and solver execution. Both phases are implemented in shell scripts, to ensure consistent and reproducible workflow.

3.1.1 Mesh generation

The first step is to create the `system/blockMeshDict` file. It is done with the python code `blockMeshDictGen.py`, that load the **`ofblockmeshdicthelper`** mentioned in the previous section.

Check that the python virtual environment is activated and run

```
python blockMeshDictGen.py
```

The file `system/blockMeshDict` has been created.

Run the script for the mesh generation

```
./genMesh.scr
```

WARNING: It the command fails, it is probable that it is not executable. Change it with

```
chmod +x ./genMesh.scr
```

Have a look at the script and discuss the meaning of each command. Check the output of the **`checkMesh`** command and visualize the mesh with **`paraFoam`**

3.1.2 Initialization and solver execution

The `runCase.scr` performs several tasks:

1. It copies the content of the folder `0.orig` to the forlde `0`.
2. It initializes the phase distribution with the command **`setFields`** and the dictionary `system/setFieldsDict`
3. It runs the **`interFoam`** solver.

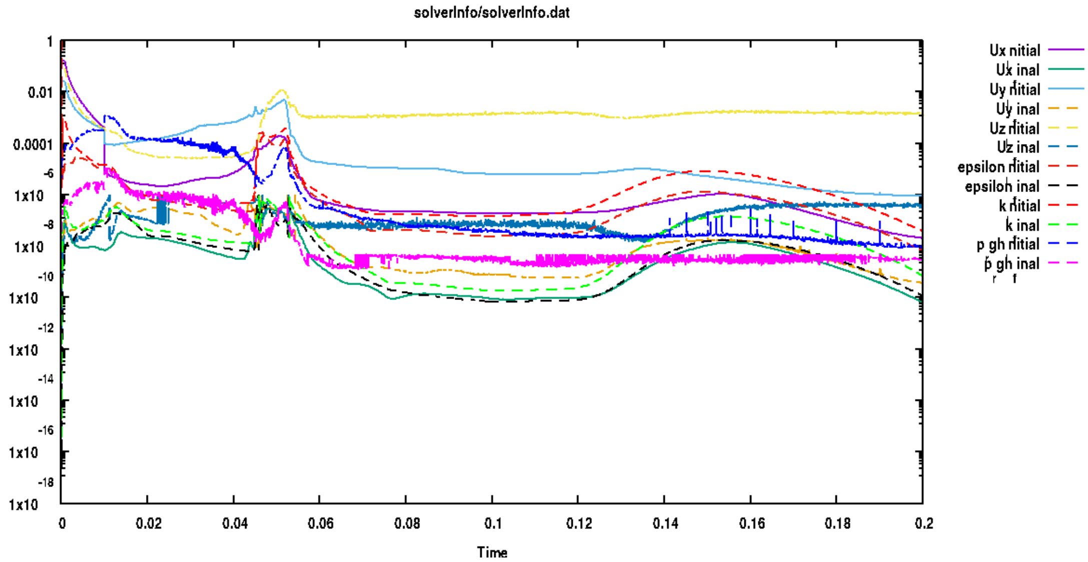
The simulation will take about 10 minutes. In the meanwhile the configuration files of the case can be studied:

- `system/controlDict` and functions definitions
- `0.orig/*`
- `constant/*`
- `system/fvSolution` and `system/fvSchemes`

3.2 Post-Processing in Terminal

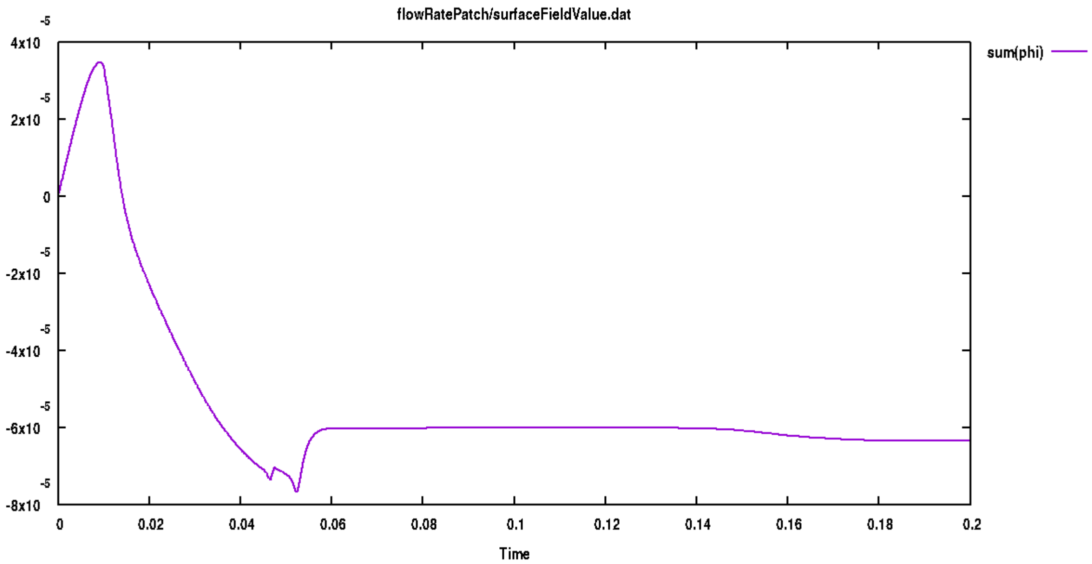
Residuals of the simulation can be check with

```
foamMonitor -l postProcessing/solverInfo/0/solverInfo.dat
```

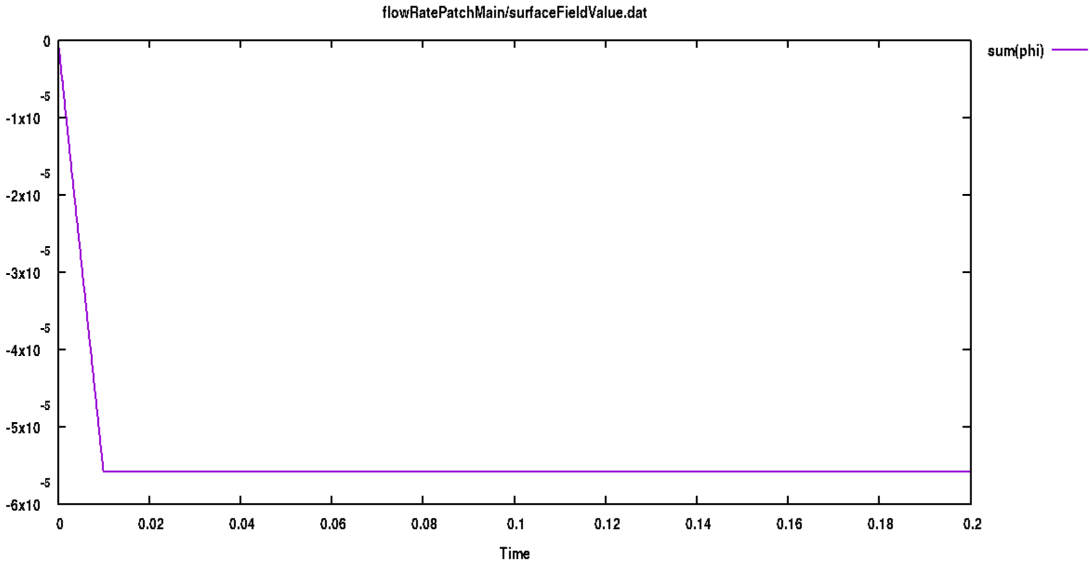


Also flow rates from main and secondary inlets

```
foamMonitor postProcessing/flowRatePatch/0/surfaceFieldValue.dat
```




```
foamMonitor postProcessing/flowRatePatchMain/0/surfaceFieldValue.dat
```



Note that the flow rate in main secondary inlets are[1]

$$
\begin{aligned}
Q_s &= 6.35 \times 10^{-5}\,\mathrm{m}^3/\mathrm{s} \\
Q_m &= 5.58 \times 10^{-5}\,\mathrm{m}^3/\mathrm{s}
\end{aligned}
$$

According to that the dimensionless **entrainment ratio** is

$$
M = \frac{Q_s}{Q_m} = 1.14
$$

Pressure in the inlet patch can also be visualized (do it by yourself)

3.3 Visualization in paraview

Progress of water and air entrainment in the ejector can be seen in paraview, and even an animation can be created with the `foamCreateVideo` utility. Phase fraction and velocity distributions are shown in Figures 1 and 2.

[1]These are the flow rate for **wedge** of the mesh. For the complete flow rate, these numbers have to be multiplied by the proportionality factor od total and wedge sections.

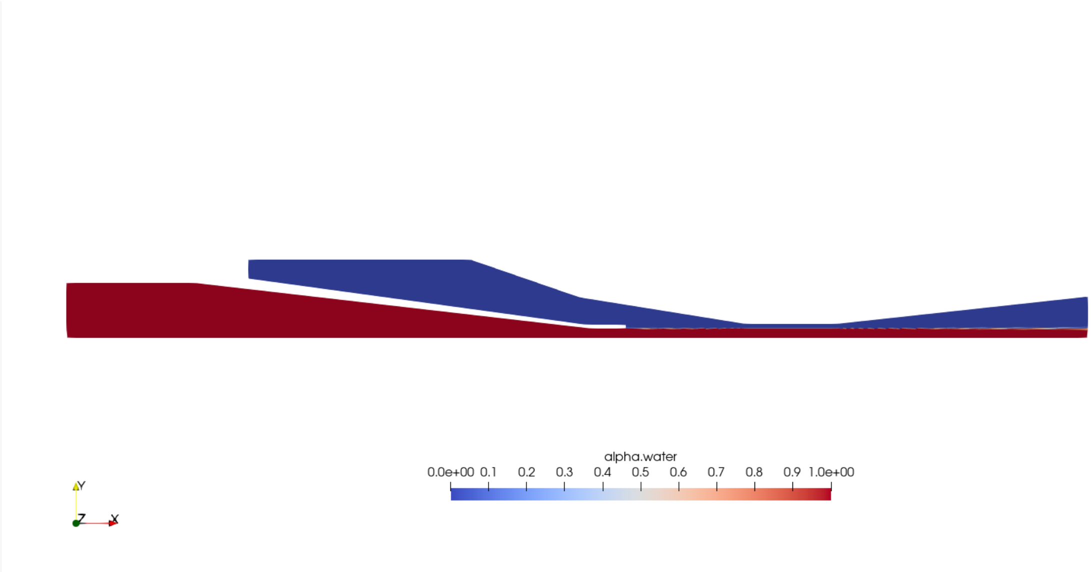


Figure 1: Water-air fraction distribution for $t = 0.2\,\mathrm{s}$

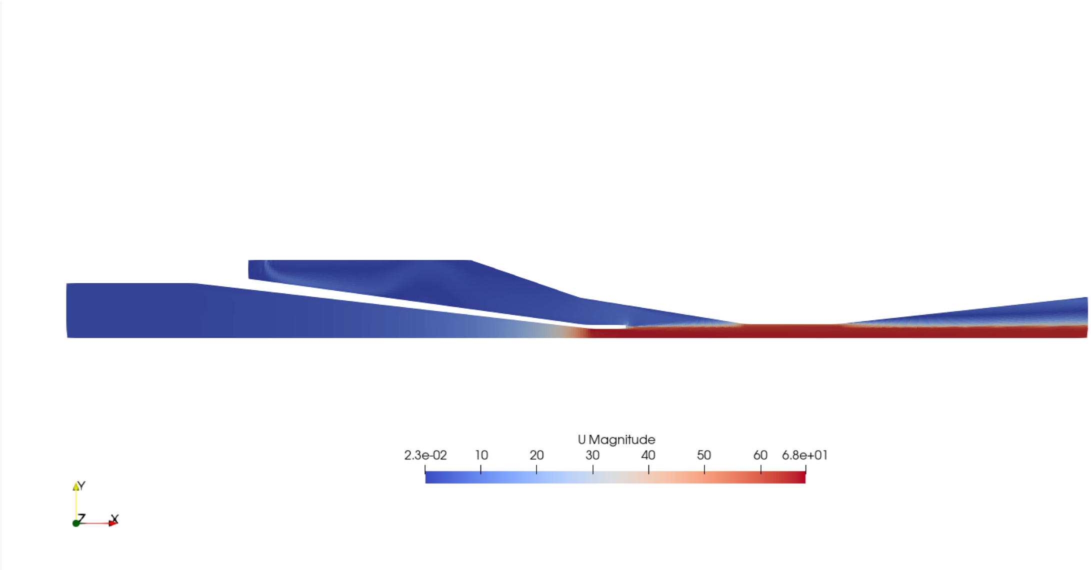


Figure 2: Velocity distribution for $t = 0.2\,\mathrm{s}$.

4 Part B: Euler-Euler Approach

The case can be also downloaded from the GitHub repository.

4.1 Running the simulation

The step for mesh generation, setup and running of the solver are exactly the same. There are some key differences on the setup.

- **Phase interaction description**. In VOF there is only a configuration file with surface tension. In Euler-Euler the interaction between forces has to be completely modeled through the `constant/phaseProperties` dictionary. For a detailed description of all the items, the great Master Thesis by Otromke[2] can be consulted.
- **Fields files and initial conditions**. The main characteristic of Euler-Euler approach is that fluid flow equations are **solved for both fluids**. That mains that two sets of field files are needed. For instance, there is a `0/U.water` file but also a `0/U.air` file.
- **Turbulence modeling**. Likewise, two turbulence modeling files are needed, one for each phase. They can be different, but compatible.

4.2 Post-Processing in Terminal

The command to see the residuals is the same as with the VOF solver

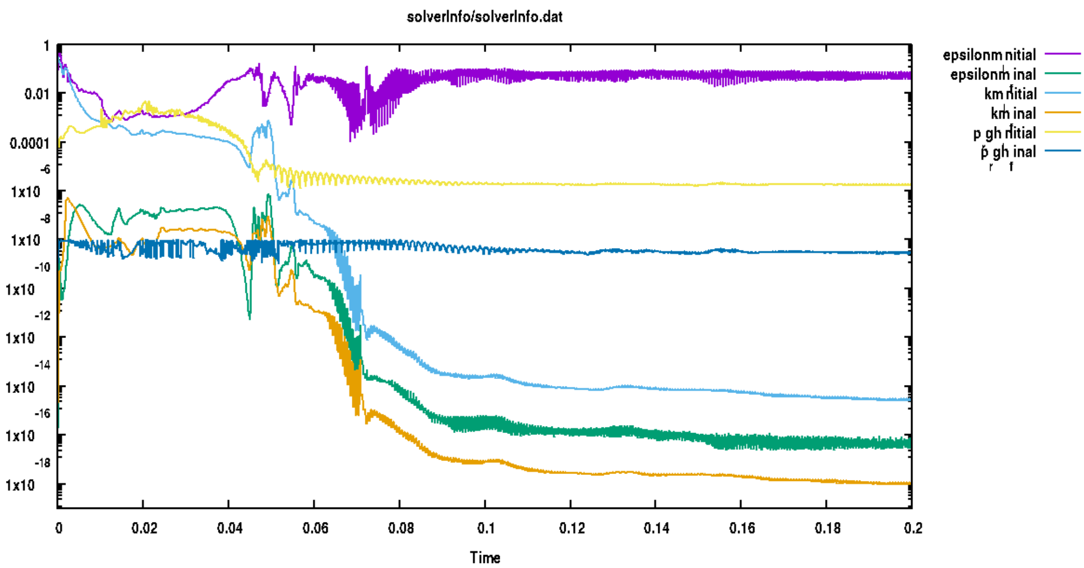


Figure 3: Residuals for the Euler-Euler approach.

[2]Otromke, Malte. 2013. "Implementation and Comparison of Correlations for Interfacial Forces in a Gas-Liquid System within an Euler-Euler Framework."

Follow the plots for flow rates in main and secondary patches.

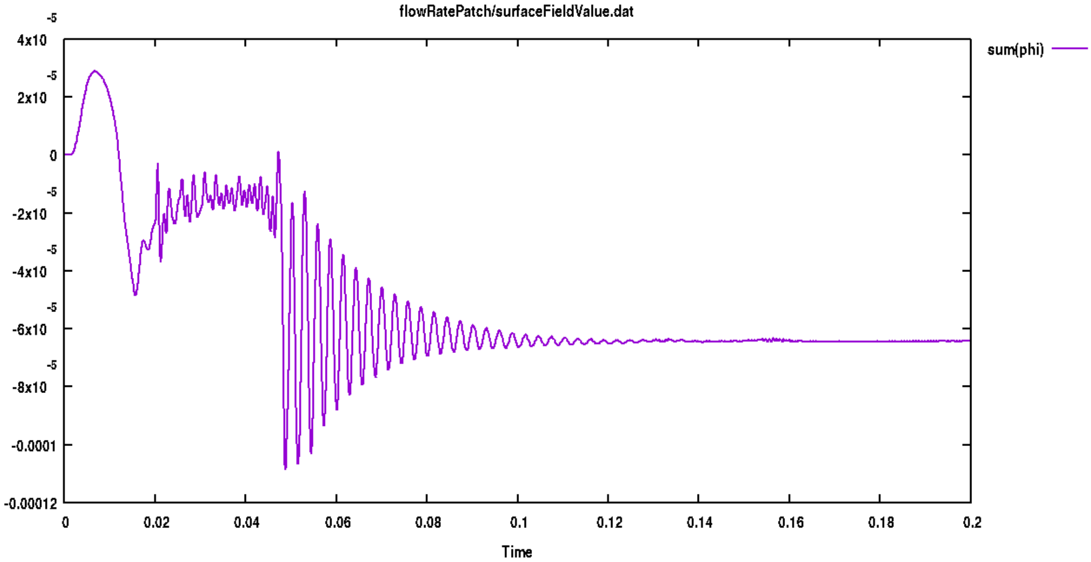


Figure 4: Flow rate in the secondary inlet.

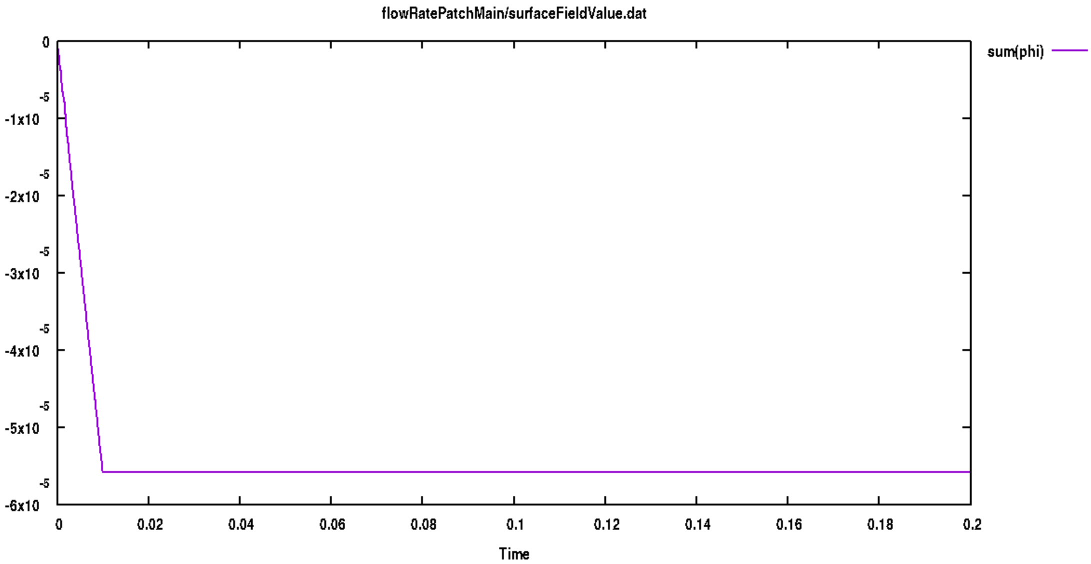


Figure 5: Flow rate in the main inlet

Note that the entrainment ratio computation gives a very similar value to that of VOF simulation

4.3 Visualization with paraview

Let's make a couple of filters to make better visualization. First, **`Reflect`** filter is used to show the complete section of the ejector. Figure 6 show the fraction of water.

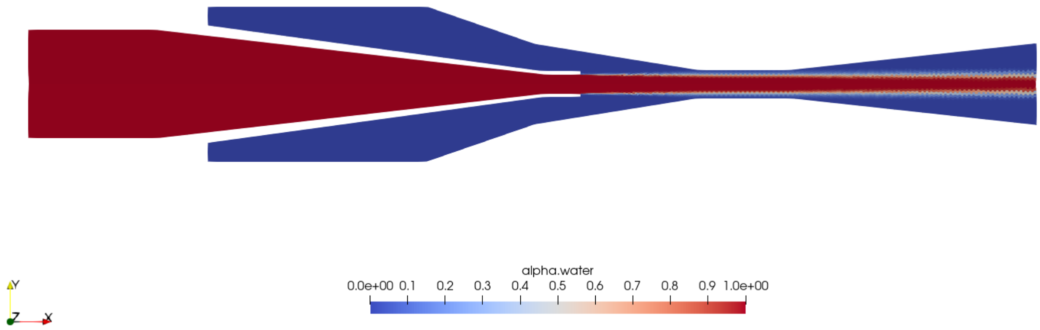
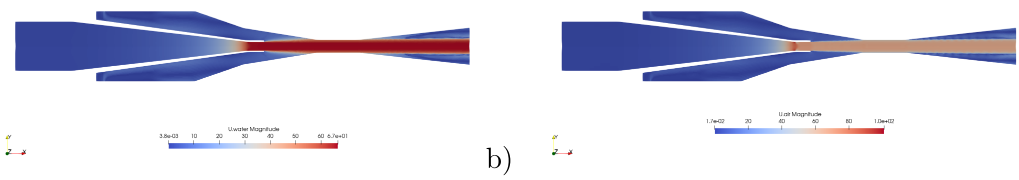


Figure 6: Volume fraction of water for the Euler-Euler approach

Secondly, to see velocity we have to decide between water (Figure 7.a) and air (Figure 7.b).

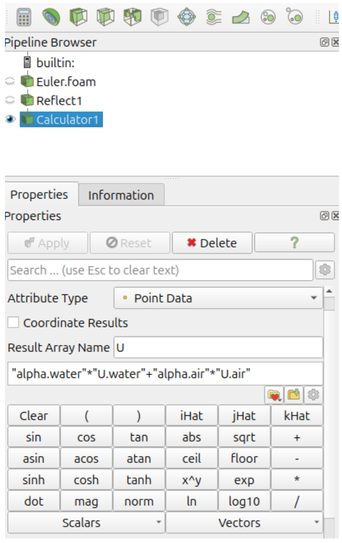


a) b)

Figure 7: Velocity distribution for water and air phases.

But they can be combined by using the **`Calculator`** filter and entering the expression

$$\mathbf{U} = \alpha_w \mathbf{U}_w + \alpha_a \mathbf{U}_a \tag{4}$$



Velocity distribution shown in Figure 8 is obtained, which is quite similar to the one in Figure 2.

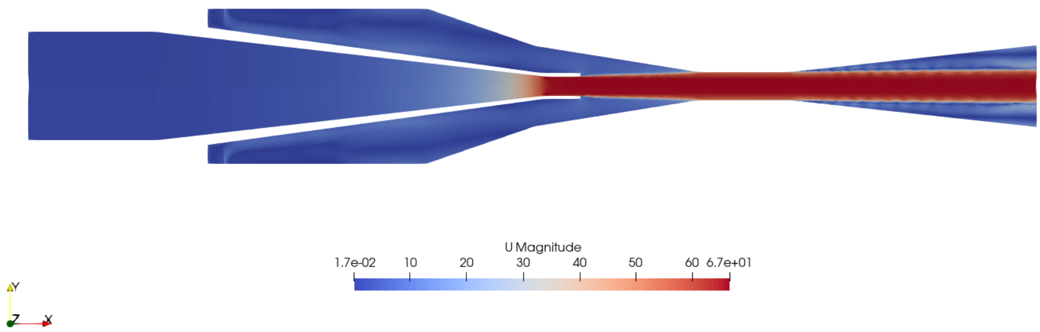


Figure 8: Combined velocity of both phases.

5 Student Exercises and Extensions

5.1 Basic exercises for Part A: VOF

1. Effect of Inlet Velocity

Modify the water inlet velocity from 2 m/s to 4 m/s. How does entrainment ratio change? Is the relationship linear?

2. Effect of Surface Tension

   Set surface tension to zero. How does the interface shape change? How does M change?

5.2 Basic exercises for Part B: Euler-Euler

1. Drag Model comparison

   Change drag model for **(air in water)** from **SchillerNaumann** to **Tomiyama-Correlated**. A value of 16 will be need in the keyword **A** (see Master Thesis by Otromke)

2. Turbulent Dispersion

   In this tutorial, turbulent dispersion has not been considered. However, it is an important factor for high Reynolds number flows. Include the usual Burn's model, with a Schmidt number $\sigma = 1.0$.

Disclaimer

This tutorial has been developed by **Robert Castilla**, of the Departament of Fluid Mechanics in the Universitat Politècnica de Catalunya, for the doctoral workshop at **IMPPAN Gdańsk** (July 2026). The author utilized **DeepSeek** for the initial definition of the document format. Further technical refinement and the development of advanced post-processing exercises were supported by **NotebookLM**. All the technical information for the **OpenFOAM v2512** environmnet and all physical interpretations were performed and approved by the author.